Week 10 Workshop Relevance (Pseudo) Models vs. IR model

IFN647

School of Computer Science

Queensland University of Technology

Workshop Objectives

- To understand how to design and implement a pseudo relevance model for a given initial query;
- To select and implement a baseline model (e.g., a BM25-based IR model);
- To evaluate the effectiveness of the proposed approach using appropriate metrics.

Workshop Task 1

In the Week 9 workshop, we discussed a **relevance model** to build an **information filtering** system. It selects **features** to represent the user **information needed** in a file "Model_w5_R102.dat" and then uses the selected features to rank documents in Test_set.

Training Set (Labelled)

w5 score for features **(did not use Q)** Top Features

```
rapist 106.20000000000002
multipl 39.77987421383647
unemploy 37.85714285714286
site 35.98159509202454
convict 31.774193548387096
featur 30.62130177514793
mistak 22.48603351955307
unknown 20.96685082872928
present 19.48087431693989
grief 18.027027027027028
defend 16.604278074866308
father 15.8064
```

Rank

Test Set

In some real applications, it is hard to get a training set. However, we can use **Pseudo Relevance feedback** as we mentioned in the lecture notes.

Training Set (Unlabelled)

BM25 score for docs (**Use Q**) Select Top Documents

BaselineModel_R102.dat

```
73038 5.898798484774149
26061 4.273638903483098
65414 4.1414522450167475
57914 3.967136888209526
58476 3.708467957856744
76635 3.5867337114200843
12769 3.4341129093591456
12767 3.352170358051889
```

Pseudo-Relevance Hypothesis: "top-ranked documents (e.g., documents' BM25 scores are great than 1.00) are possible relevant".

Output File: The labelled training document set D

PTraining_benchmark.txt

Workshop Task 1

Task 1. Design a Pseudo Relevance Model to Rank documents using an initial query Q and generate a training set.

Relevance Model (RM) doesn't have a query, doesn't generate a training set, generates features
Pseudo Relevance Model (PRM) has a query, generates a training set

Workshop Task 1

(1) Design python function *bm25*(*coll*, *q*, *df*) to calculate BM25 score for all documents in the "Training_set", where *coll* is the output of *coll.parse_rcv_coll*(*coll_fname*, *stop_words*), see week 9 solution if you do not know this function; *q* is a query (e.g., q = "Convicts, repeat offenders"), and *df* is a dictionary of term document-frequency pairs.

(2) Call function *bm25*() in the main function and save the result into a text file, PRModel_R102.dat, in which each row includes the document number and the corresponding BM25 score, and sorted it in descendent order.

```
73038 5.898798484774149
26061 4.273638903483098
65414 4.1414522450167475
57914 3.967136888209526
58476 3.708467957856744
76635 3.5867337114200843
12769 3.4341129093591456
12767 3.352170358051889
```

(3) Extend the main function to generate a training set $\underline{D}$ which includes both $\underline{D}^+$ (positive – likely relevant documents) and $\underline{D}^-$ (negative – likely irrelevant documents) in the given un-labelled document set *U* (e.g., *U* = Training_set). The output of this function is a file "PTraining_benchmark.txt", which has the following sample output:

```
R102 73038 1
R102 26061 1
R102 65414 1
R102 57914 1
...
R102 86912 0
R102 86929 0
R102 11922 0
...
```

Week 8 Workshop Task

Task 2. Training set generation

Pseudo-Relevance Hypothesis: "top-ranked documents (e.g., documents' BM25 scores are great than 1.00) are possible relevant".

Design a python program to find a training set $\underline{D}$ which includes both $\underline{D}^+$ (positive – likely relevant documents) and $\underline{D}^-$ (negative – likely irrelevant documents) in the given un-labelled document set *U*.

The input file is "BaselineModel_R102.dat" or BaselineModel_R102_v2.dat, and
The output file name is "PTraining_benchmark.txt", which has the following sample output:

Workshop Task 1

(4) Re-run the week 9 solution (the two .py files) by replacing "Training_benchmark.txt" with "PTraining_benchmark.txt", you may get the following output:

At position 1, precision= 1.0, recall= 0.034482758620689655
At position 2, precision= 1.0, recall= 0.06896551724137931
At position 3, precision= 1.0, recall= 0.10344827586206896
At position 4, precision= 1.0, recall= 0.13793103448275862
At position 5, precision= 1.0, recall= 0.1724137931034483
At position 10, precision= 0.6, recall= 0.20689655172413793
At position 11, precision= 0.6363636363636364, recall= 0.2413793103448276
At position 12, precision= 0.6666666666666666, recall= 0.27586206896551724
At position 13, precision= 0.6923076923076923, recall= 0.3103448275862069
At position 14, precision= 0.7142857142857143, recall= 0.3448275862068966
At position 16, precision= 0.6875, recall= 0.3793103448275862
At position 17, precision= 0.7058823529411765, recall= 0.41379310344827586
At position 18, precision= 0.7222222222222222, recall= 0.4482758620689655
At position 19, precision= 0.7368421052631579, recall= 0.4827586206896552
At position 20, precision= 0.75, recall= 0.5172413793103449

Week9 IF

Last week : use Training_benchmark.txt to calculate w5 scores for features in training doc and then select top features, use those top features to rank test doc

Today: same thing using PTraining_benchmark.txt

Workshop Task 2

Task 2. Design a BM25 based IR model.

This task needs to rank documents in “Test_set” **directly** by using function bm25(coll, q, df), and save the ranking result in IRModel_R102. Then test the ranking result using the similar evaluation methods as we did in week 9 (see “test_eval_bm25_wk9.py”), and you may get the following output:

```
At position 2, precision= 0.5, recall= 0.034482758620689655
At position 3, precision= 0.6666666666666666, recall= 0.06896551724137931
At position 5, precision= 0.6, recall= 0.10344827586206896
At position 9, precision= 0.4444444444444444, recall= 0.13793103448275862
At position 10, precision= 0.5, recall= 0.1724137931034483
At position 13, precision= 0.46153846153846156, recall= 0.20689655172413793
At position 14, precision= 0.5, recall= 0.2413793103448276
At position 15, precision= 0.5333333333333333, recall= 0.27586206896551724
At position 16, precision= 0.5625, recall= 0.3103448275862069
At position 17, precision= 0.5882352941176471, recall= 0.3448275862068966
```

The difference with Task 1: PRM vs. IRM

In PRM,

training set → BM25 threshold to give label to train doc→ top features of those relevant doc → rank test data

IN IRM,

No training set; directly use query and test data to do the ranking and evaluation

All the documents in the ranking file are retrieved (we can specify top k doc)

Workshop Task 3

Task 3. Analyse Your Pseudo Relevance Model.

So far, we have discussed three IF models: **the relevance model** (**RM**, see week 9 workshop), **the pseudo relevance model** (**PRM**, Task 1) and **the IR model** (**IRM**, Task 2), for ranking documents in **Test_set** (a set of unlabelled data). Table 1 shows the experimental results.

Table1. The experimental results over average precision

Topic	RM	IRM	PRM
R102	1.00	0.6625	0.7973
...			

RM (last week)	Training set (labelled)→ w5 scores for features → Top Features → calculate rank score for test documents
PRM(task 1)	Training set (unlabelled) → BM25 scores for train doc → select relevant train doc according to BM25 threshold and give the label → w5 scores for features → Top Features → calculate rank score for test documents
IR(task 2)	Test data set → calculate BM25 score according to query → rank test documents

Workshop Task 3 cont.

Based on the average precision for topic R102, the pseudo relevance model outperforms the IR model, but worse than the relevance model.

You may further refine the pseudo relevance model through a validation process. For example, you can tune the BM25 threshold parameter or modify the BM25 formulation to investigate whether improved performance can be achieved.

You are encouraged to discuss your validation strategy with classmates or your tutor. In particular, you may apply a parameter search method (e.g., grid search) to determine appropriate parameter values. For more details, refer to the Wikipedia page on hyperparameter optimization: https://en.wikipedia.org/wiki/Hyperparameter_optimization